

Introduction to Probabilistic Machine Learning

Linear Basis Function Models

Ralf Herbrich

Overview

1. Modelling Data
 - Modelling Text
 - Modelling Images
2. Linear Basis Function Models
 - Vector Spaces
 - Linear Mappings and Matrices

**Introduction to
Probabilistic Machine
Learning**

*Unit 7 – Linear Basis Function
Models*

1. Modelling Data

- **Modelling Text**
- Modelling Images

2. Linear Basis Function Models

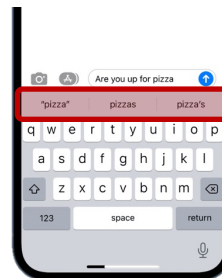
- Vector Spaces
- Linear Mappings and Matrices

**Introduction to
Probabilistic Machine
Learning**

*Unit 7 – Linear Basis Function
Models*

Modelling Text

- Text can be modelled at three levels of granularity:
 1. **Letters** ($\approx 10^2$ different letters in most alphabets)
 2. **Tokens** (10^3 to 10^4 different tokens in most alphabets)
 3. **Words** (10^5 to 10^6 different words in most languages)
- Modelling level of granularity depends on the application
 1. **Letters:** Compression algorithms for textual data
 2. **Tokens:** Sequence prediction for question-answering
 3. **Words:** Auto-correct function in smart keyboards



Introduction to Probabilistic Machine Learning

Unit 7 – Linear Basis Function Models

One-Hot Encoding

- Text is a sequence of text elements $s_1, s_2, s_3, \dots$
- We often want to know the importance of each text element s_i to the target
 - Example.** "This product shipped fast but was disappointing" has negative sentiment

s_1 s_2 s_3 s_4 s_5 s_6 s_7

Text element that is likely the expression of bad sentiment

- One-Hot Encoding.** Given a dictionary S and a text element $s \in S$, a one-hot encoding $\phi_{\text{OHE}}(s)$ is an $|S|$ -dimensional unit vector indexed by the elements of S that consists of 0s in all dimensions with the exception of a single 1 in the dimension indexed by s .

- Example (ctd).** If we assume the indices are $s_1, s_2, \dots, s_7$ then

$$\phi_{\text{OHE}}(\text{"fast"}) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{array}{l} \leftarrow \text{This} \\ \leftarrow \text{product} \\ \leftarrow \text{shipped} \\ \leftarrow \text{fast} \\ \leftarrow \text{but} \\ \leftarrow \text{was} \\ \leftarrow \text{disappointing} \end{array}$$

Introduction to
Probabilistic Machine
Learning

Unit 7 – Linear Basis Function
Models

Modelling Whole Text

- Two common techniques for encoding a whole text

- Variable-length text** $s_1 s_2 \dots$: Sum of all one-hot encoded vectors (“bag of words”)

$$\phi_{\text{BOW}}(s_1 s_2 \dots) = \sum_j \phi_{\text{OHE}}(s_j)$$

- Fixed-length text** $s_1 s_2 \dots s_n$: A stacked $n \cdot |S|$ dimensional vector

$$\phi_{\text{FL}}(s_1 s_2 \dots s_n) = \begin{bmatrix} \phi_{\text{OHE}}(s_1) \\ \vdots \\ \phi_{\text{OHE}}(s_n) \end{bmatrix}$$

- Example:** Consider the sentences

- s_1 = “This product shipped not fast and was disappointing”
 - s_2 = “This product shipped fast and was not disappointing”

$$\phi_{\text{BOW}}(s_1) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \begin{array}{l} \leftarrow \text{This} \\ \leftarrow \text{product} \\ \leftarrow \text{shipped} \\ \leftarrow \text{not} \\ \leftarrow \text{fast} \\ \leftarrow \text{and} \\ \leftarrow \text{was} \\ \leftarrow \text{disappointing} \end{array}$$

$$\phi_{\text{BOW}}(s_2) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \begin{array}{l} \leftarrow \text{This} \\ \leftarrow \text{product} \\ \leftarrow \text{shipped} \\ \leftarrow \text{not} \\ \leftarrow \text{fast} \\ \leftarrow \text{and} \\ \leftarrow \text{was} \\ \leftarrow \text{disappointing} \end{array}$$

Bag of words **cannot** learn
“positional” effect of text elements

**Introduction to
Probabilistic Machine
Learning**

*Unit 7 – Linear Basis Function
Models*

1. Modelling Data

- Modelling Text
- **Modelling Images**

2. Linear Basis Function Models

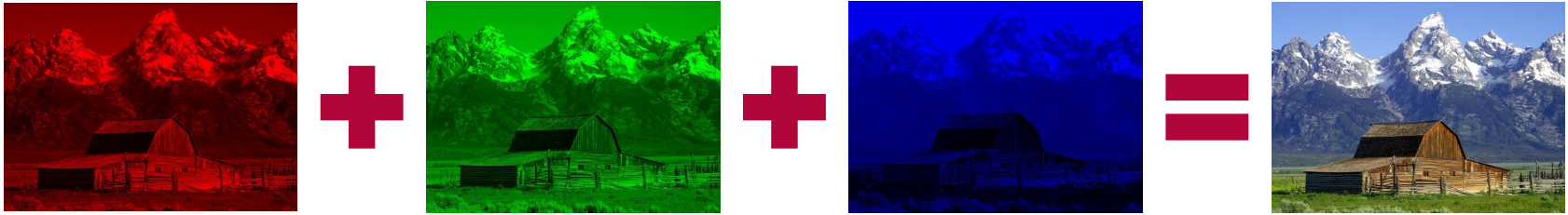
- Vector Spaces
- Linear Mappings and Matrices

**Introduction to
Probabilistic Machine
Learning**

*Unit 7 – Linear Basis Function
Models*

Modelling Images

- **Raw image data.** *An image is a rectangular array of picture elements ("pixels") that consist of a triple of intensities of the base colors red, blue, and green.*



- Images can also be modelled at three levels of granularity depending on application

1. **Pixels:** Image segmentation
2. (Non-overlapping) **patches:** Object recognition
3. **Whole image:** Image classification

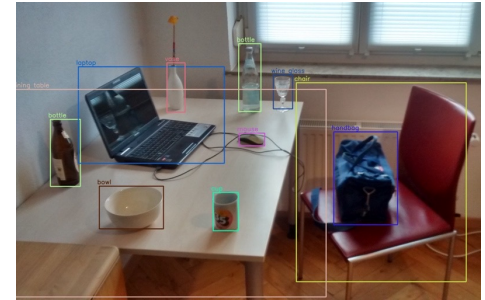


Image Content in Frequency and Location

- Image data contains signal (photon counts) at *fixed* locations in the image

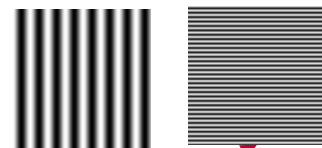
$$\phi_{\text{RGB}} \left(\text{Image} \right) = \begin{bmatrix} 127 \\ 0 \\ \vdots \\ 32 \\ 6 \end{bmatrix}$$

Red value at location (1,1)
 Green value at location (1,1)
 Green value at location (800,600)
 Blue value at location (800,600)

- Observation.** Recurring patterns are more visible in the frequency domain.
- Idea.** Discrete Fourier transform (DFT) to transform the image

$$\phi_{\text{DFT}}(x) = F_{\text{DFT}} \phi_{\text{RGB}}(x)$$

- In practice, there is a $\mathcal{O}(\#\text{pixel} \cdot \log(\#\text{pixel}))$ algorithm for fast Fourier transform (FFT)

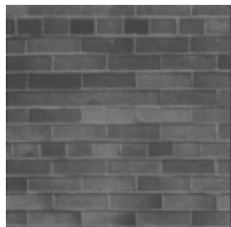


$$F_{\text{DFT}} = W_{\text{DFT}} \otimes W_{\text{DFT}}$$

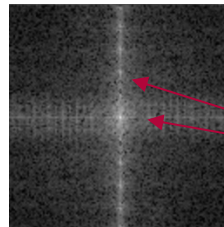
Kronecker product of 1D discrete Fourier transform W_{DFT}

Introduction to Probabilistic Machine Learning

Unit 7 – Linear Basis Function Models



Raw image (in single-grey channel)

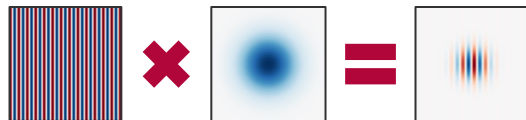


Fourier transform of image

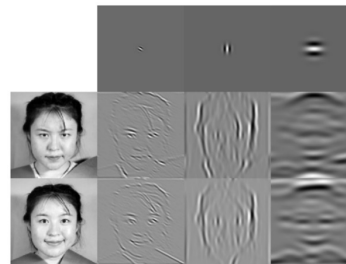
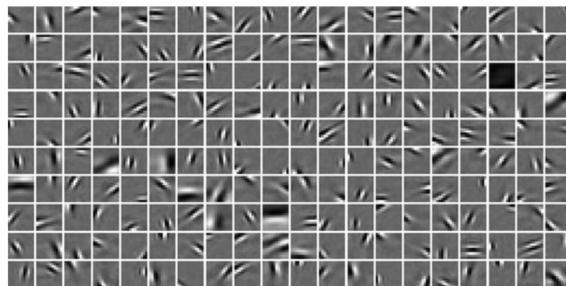
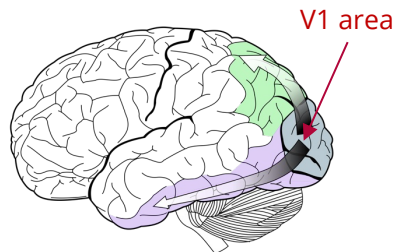
Horizontal and vertical patterns by single features

Gabor Wavelets: Mixing Location and Frequency

- Both representations are extremes:
 - **Raw image:** Single features $\phi_{\text{RGB},i}(x)$ at a location without frequency
 - **DFT image:** Single features $\phi_{\text{DFT},i}(x)$ in frequency without a location
- **Gabor wavelets.** A *Gabor wavelet* is a set of basis functions that is obtained by multiplying a Fourier basis function with a Gaussian density.



- Combine both spatial and frequency information in image features
- Basis functions for sparse encoding by the brain in V1



Introduction to
Probabilistic Machine
Learning

Unit 7 – Linear Basis Function
Models

Filters “implemented” in the V1
(sparse coding!)

Bruno Olshausen & David Field (1997), [Sparse Coding with Overcomplete Basis Set: A Strategy Employed by V1?](#)



Dennis Gabor
(1900 – 1978)

1. Modelling Data

- Modelling Text
- Modelling Images

2. Linear Basis Function Models

- Vector Spaces
- Linear Mappings and Matrices

**Introduction to
Probabilistic Machine
Learning**

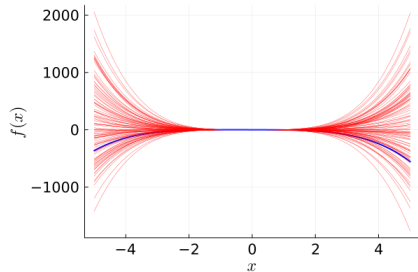
*Unit 7 – Linear Basis Function
Models*

Linear Basis Function Models

- **Linear Basis Function Models.** Given an input space $\mathcal{X}$ and D basis functions (“features”) $\phi_j: \mathcal{X} \rightarrow \mathbb{R}$, a linear basis function model is a function of the form
$$f(x; \mathbf{w}) = w_0 + w_1 \cdot \phi_1(x) + w_2 \cdot \phi_2(x) + \dots + w_D \cdot \phi_D(x)$$
- It’s called a linear model, but the linearity is w.r.t. $\mathbf{w}$ not $x \in \mathcal{X}$!
- **Examples:** 4 basis function ($D = 4$) and 100 random parameters $\mathbf{w}$.

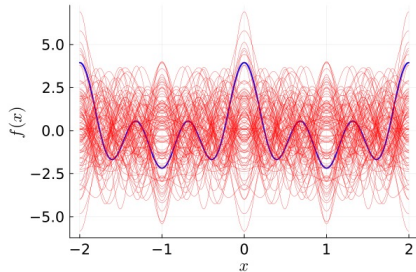
Polynomial Basis

$$\phi_j(x) = x^j$$



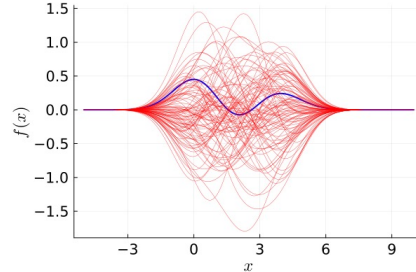
Fourier Basis

$$\phi_j(x) = \cos(\pi j \cdot x)$$



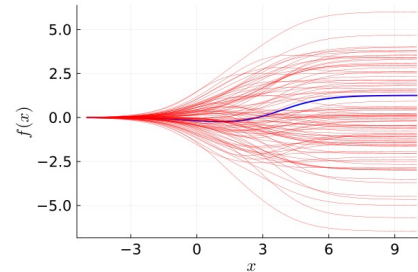
Gaussian Basis

$$\phi_j(x) = \mathcal{N}(x; j, 1)$$



Sigmoid Basis

$$\phi_j(x) = \frac{\exp(x - j)}{1 + \exp(x - j)}$$



Linear Models for Text

- A linear model captures the *effect* of presence of a text element!
- In practice, the feature vector for one-hot encoded data is *never* explicitly computed because

$$\mathbf{w}^T \boldsymbol{\phi}_{\text{OHE}}(s) = \sum_{i=1}^{|S|} w_i \cdot \phi_i(s) = w_{\text{idx}(s)}$$

$\mathcal{O}(|S|)$ (pointing to the sum)

$\mathcal{O}(\log_2 |S|)$ (pointing to the index)

- All we need is the inverse function mapping of an actual text element s
- Two common techniques for encoding a whole text

$$\text{idx}(s) := i \Leftrightarrow \phi_{\text{OHE},i}(s) = 1$$

- **Variable-length text** $s_1 s_2 \dots$: Sum of all one-hot encoded vectors (“bag of words”)

$$\boldsymbol{\phi}_{\text{BOW}}(s_1 s_2 \dots) = \sum_j \boldsymbol{\phi}_{\text{OHE}}(s_j)$$

- **Fixed-length text** $s_1 s_2 \dots s_n$: A stacked $n \cdot |S|$ dimensional vector

$$\boldsymbol{\phi}_{\text{FL}}(s_1 s_2 \dots s_n) = \begin{bmatrix} \boldsymbol{\phi}_{\text{OHE}}(s_1) \\ \vdots \\ \boldsymbol{\phi}_{\text{OHE}}(s_n) \end{bmatrix}$$

A linear model is only $\mathcal{O}(n \cdot \log_2 |S|)$

Introduction to
Probabilistic Machine
Learning

Unit 7 – Linear Basis Function
Models

Overview

1. Modelling Data
 - Modelling Text
 - Modelling Images
2. Linear Basis Function Models
 - **Vector Spaces**
 - Linear Mappings and Matrices

**Introduction to
Probabilistic Machine
Learning**

*Unit 7 – Linear Basis Function
Models*

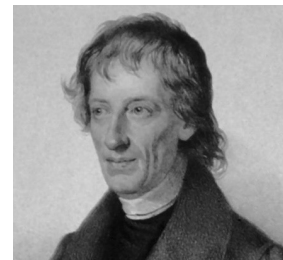
Linear Models and Vector Spaces

- To write a linear model $f(x; \mathbf{w}) = \sum_{i=1}^D w_i \cdot \phi_i(x)$ compactly, we use two *vectors*

1. A vector $\boldsymbol{\phi} = \begin{bmatrix} \phi_1(x) \\ \vdots \\ \phi_D(x) \end{bmatrix}$ of D feature values for the input x
2. A vector $\mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_D \end{bmatrix}$ of D weightings (“weight” parameters)

- **Vector Space.** A vector space is a set V that satisfies the following axioms: There exists a null element $\mathbf{0} \in V$ such that for all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$ and scalars $a, b \in \mathbb{R}$

1. **Associativity:** $\mathbf{u} + (\mathbf{v} + \mathbf{w}) = (\mathbf{u} + \mathbf{v}) + \mathbf{w}$
2. **Commutativity:** $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$
3. **Identity element (of vector addition):** $\mathbf{v} + \mathbf{0} = \mathbf{v}$
4. **Inverse element:** $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$
5. **Identity element (of scalar multiplication):** $1 \cdot \mathbf{v} = \mathbf{v}$
6. **Distributivity (w.r.t. vector addition):** $a \cdot (\mathbf{u} + \mathbf{v}) = a \cdot \mathbf{u} + a \cdot \mathbf{v}$
7. **Distributivity (w.r.t. scalar addition):** $(a + b) \cdot \mathbf{v} = a \cdot \mathbf{v} + b \cdot \mathbf{v}$



Bernhard Bolzano
(1781 – 1848)

**Introduction to
Probabilistic Machine
Learning**

*Unit 7 – Linear Basis Function
Models*

Linear Independence, Span and Basis

- **Linear combination.** Given a set $v_1, v_2, \dots, v_n$ of a vector space V , a linear combination is defined by ($a_1, a_2, \dots, a_n$ are called coefficients)

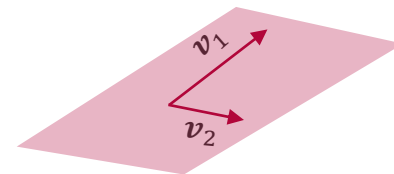
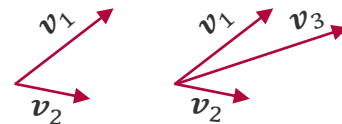
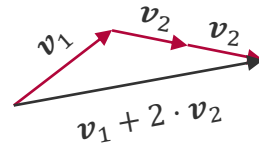
$$a_1 \cdot v_1 + a_2 \cdot v_2 + \dots + a_n \cdot v_n$$

- **Linear independence.** A set $v_1, v_2, \dots, v_n$ is called linearly independent if no v_i can be written as a linear combination of $v_1, \dots, v_{i-1}, v_{i+1}, \dots, v_n$.
- **Span.** Given a set $v_1, v_2, \dots, v_n$ of a vector space V , the span $\text{span}(v_1, v_2, \dots, v_n)$ is the set of all linear combinations of $v_1, v_2, \dots, v_n$.

□ **Example:**

- $\text{span}(v_1)$ is the line through the origin along v_1
- $\text{span}(v_1, v_2)$ is the plane through the origin along v_1 and v_2

- **Basis.** A subset $b_1, b_2, \dots, b_n$ of a vector space V is called a basis if its span equals the whole vector space, that is, $\text{span}(b_1, b_2, \dots, b_n) = V$.
- **Dimensionality.** All bases of a vector space V have the same cardinality called the dimensionality of the vector space, $\dim(V)$.

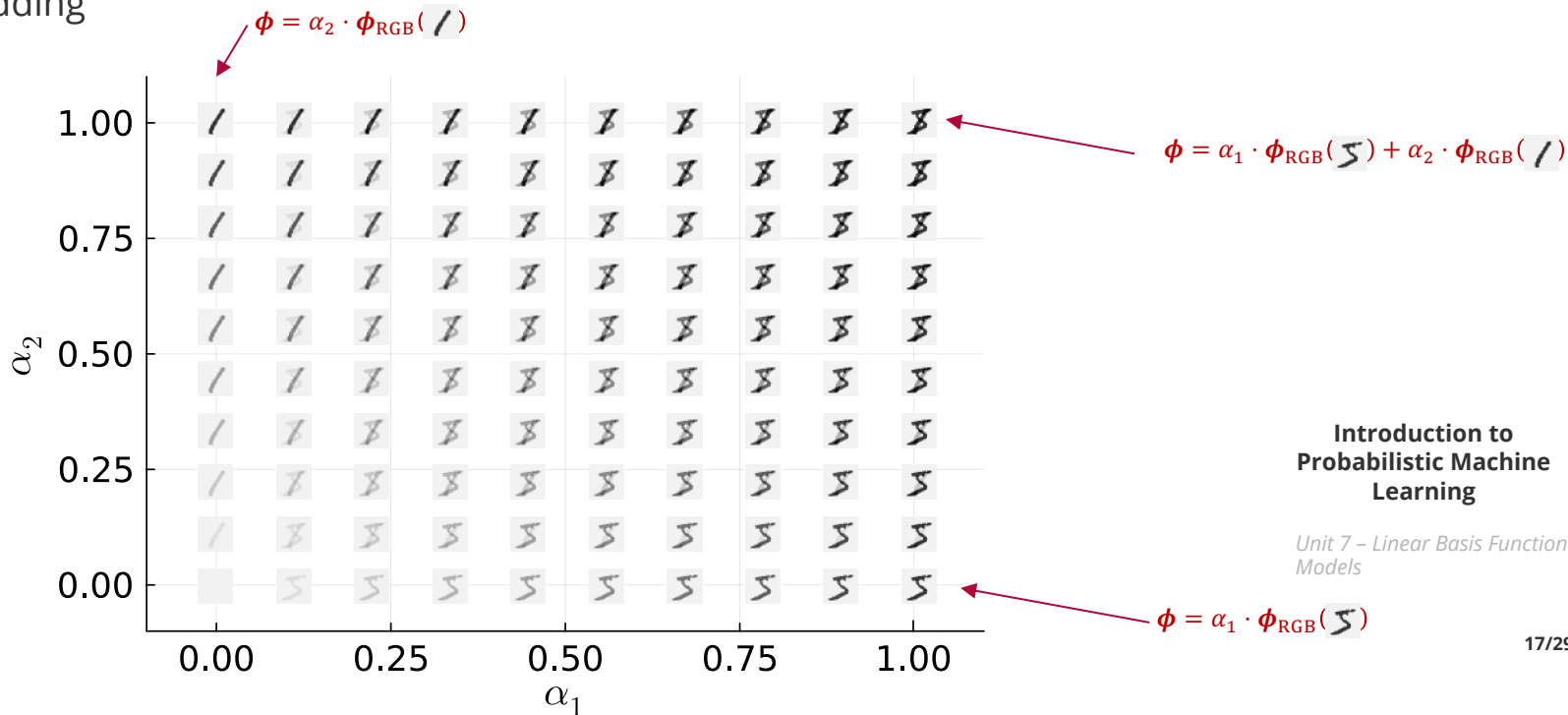


**Introduction to
Probabilistic Machine
Learning**

Unit 7 – Linear Basis Function
Models

Example: Linear Independence & Span

- Consider two 28x28 pictures Σ and $/$ and their $28^2 = 784$ -dimensional ϕ_{RGB} embedding



**Introduction to
Probabilistic Machine
Learning**

Unit 7 – Linear Basis Function
Models

Distances and Scalar Products

- So far, a vector space is a set but there is no notion of **distance**!
 - In New York, the **walking distance** between two points is much longer than the **flying distance**!
- **Metric space.** A metric space is a vector space V together with a metric $d: V \times V \rightarrow \mathbb{R}^+$ that has the following properties for all x, y, z :

$$d(x, y) = 0 \Leftrightarrow x = y$$

$$d(x, y) = d(y, x) \quad \leftarrow \text{symmetry}$$

$$d(x, y) \leq d(x, z) + d(z, y) \quad \leftarrow \text{triangle inequality}$$

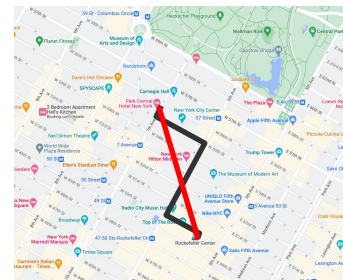
- **Scalar product.** Given a vector space a scalar product $x^T y \in \mathbb{R}$ has the following properties for all x, y, z :

$$x^T x = 0 \Leftrightarrow x = 0$$

$$x^T y = y^T x \quad \leftarrow \text{symmetry}$$

$$(a \cdot x + b \cdot y)^T z = a \cdot x^T z + b \cdot y^T z \quad \leftarrow \text{linearity}$$

- Given a scalar product, the function $d(x, y) = \sqrt{(x - y)^T (x - y)}$ is a metric!



**Introduction to
Probabilistic Machine
Learning**

Unit 7 – Linear Basis Function
Models

1. Modelling Data

- Modelling Text
- Modelling Images

2. Linear Basis Function Models

- Vector Spaces
- **Linear Mappings and Matrices**

**Introduction to
Probabilistic Machine
Learning**

*Unit 7 – Linear Basis Function
Models*

Linear Mappings and Matrices

- **Linear mapping.** A function $f: V \rightarrow W$ is a linear mapping between vector space V and W if for any two vectors $\mathbf{u}, \mathbf{v} \in V$ and any scalar $c \in \mathbb{R}$:

$$f(\mathbf{u} + \mathbf{v}) = f(\mathbf{u}) + f(\mathbf{v})$$

$$f(c \cdot \mathbf{u}) = c \cdot f(\mathbf{u})$$

- Addition and scalar multiplication can be applied before or after the map!
- It follows that $f(\mathbf{0}) = f(0 \cdot \mathbf{v}) = 0 \cdot f(\mathbf{v}) = \mathbf{0}$!

- **Theorem.** Every linear mapping $f: V \rightarrow W$ between V and W of dimension n and m can be represented via a matrix multiplication with an $m \times n$ matrix $\mathbf{A}$. The rank of $\mathbf{A}$ is the dimensionality of the image of W , that is $\text{rank}(\mathbf{A}) = \dim(W)$.

- **Proof.** If $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ is a basis for V and $\{\mathbf{w}_1, \dots, \mathbf{w}_m\}$ is a basis for W then we know

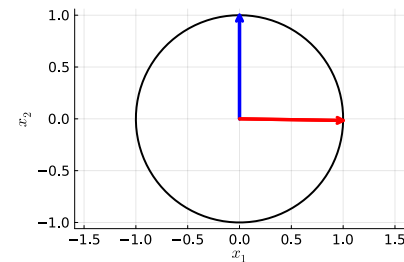
because $\mathbf{v}_1, \dots, \mathbf{v}_n$ is a basis $f(\mathbf{v}_j) = a_{1,j} \cdot \mathbf{w}_1 + \dots + a_{m,j} \cdot \mathbf{w}_m$ because $\mathbf{w}_1, \dots, \mathbf{w}_m$ is a basis

$f(\mathbf{v}) = f(c_1 \mathbf{v}_1 + \dots + c_n \mathbf{v}_n) = c_1 \cdot f(\mathbf{v}_1) + \dots + c_n \cdot f(\mathbf{v}_n)$ because f is a linear mapping

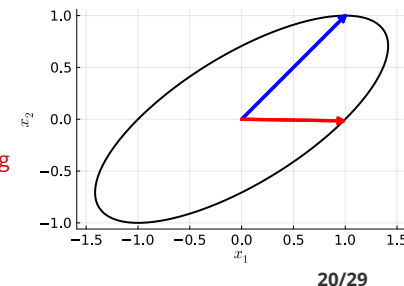
$$= \left(\sum_{j=1}^n c_j \cdot a_{1,j} \right) \cdot \mathbf{w}_1 + \dots + \left(\sum_{j=1}^n c_j \cdot a_{m,j} \right) \cdot \mathbf{w}_m$$

$[A\mathbf{c}]_m$

- For $V = \mathbb{R}^n$ and $W = \mathbb{R}^m$, the columns of $\mathbf{A}$ are the images of the basis vectors in $\mathbb{R}^n$.

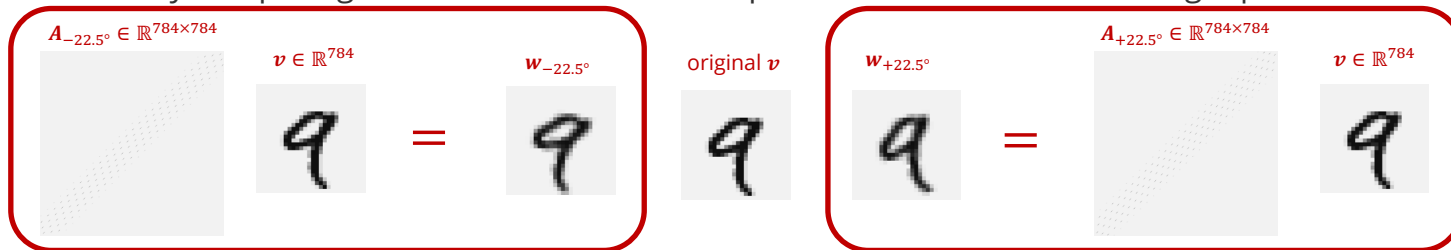


$$f(\mathbf{x}) = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \mathbf{x}$$



Example: Linear Mappings and Matrices

- Consider a 28x28 picture q and its $28^2 = 784$ -dimensional ϕ_{RGB} embedding
- Then the rotation of the image content can be expressed as a linear mapping from 784-dimensional onto itself
 - By computing the four fractional source pixel that contributed to a target pixel



- The following movie was produced by 120 linear mappings of 3° applied to the same 784-dimensional ϕ_{RGB} embedding of the source image



Matrix Types and Properties

- **Square Matrix.** A matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$ where $m = n$ is called a square matrix.
 - It parameterizes mappings of a space onto itself.
- **Diagonal Matrix.** A square matrix $\mathbf{A}$ is called diagonal if $\mathbf{A}_{ij} = 0$ for all $i \neq j$.
 - Geometrically, a diagonal matrix is an axis scaling (mapping).
 - A special diagonal matrix is $\mathbf{A}_{ii} = 1$ which is also called identity matrix $\mathbf{I}$.
- **Orthogonal Matrix.** A square matrix $\mathbf{A}$ is called orthogonal if $\mathbf{A}\mathbf{A}^T = \mathbf{I}$.
 - Geometrically, an orthogonal matrix is a rotation and mirroring (mapping).
- **Symmetric Matrix.** A square matrix $\mathbf{A}$ is called symmetric if and only if $\mathbf{A} = \mathbf{A}^T$.
- **Positive semi-definite matrix.** A symmetric matrix $\mathbf{A}$ is called positive definite if and only if

$$\forall \mathbf{x} \neq \mathbf{0}: \mathbf{x}^T \mathbf{A} \mathbf{x} > 0$$

- Positive definiteness means that no axis is inverted or removed in the mapping

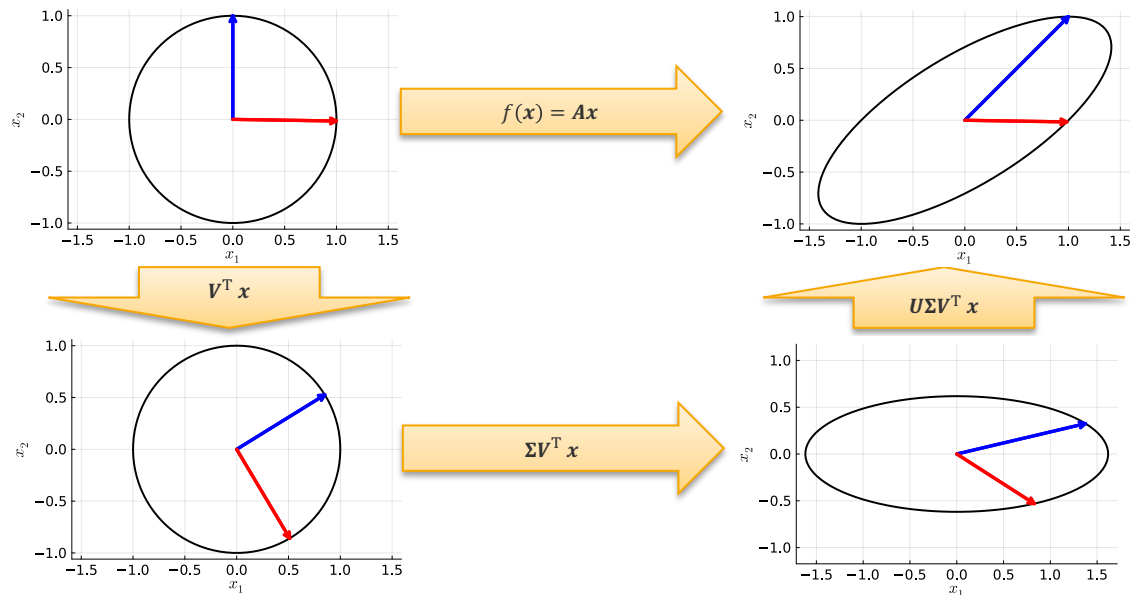
**Introduction to
Probabilistic Machine
Learning**

Unit 7 – Linear Basis Function
Models

Singular Value Decomposition

- **Singular Value Decomposition.** Any matrix $A \in \mathbb{R}^{n \times m}$ has a decomposition into three matrices $U \in \mathbb{R}^{n \times k}$, $\Sigma \in \mathbb{R}^{k \times k}$ and $V \in \mathbb{R}^{m \times k}$ such that $U^T U = I$ and $V^T V = I$ and Σ is only non-zero on diagonal elements (where $k = \text{rank}(A)$)

$$A = U \Sigma V^T$$



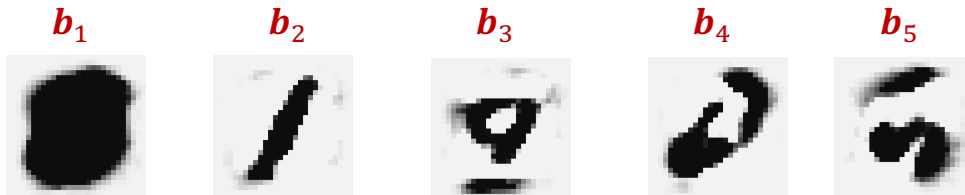
Singular Value Decomposition Example

- Imagine we arrange the 784-dimensional RGB vectors for 1000 images of digits in the rows of a matrix A . Then the SVD has the following property

$$A = U \Sigma V^T = UB = [u_1 \ \cdots \ u_{784}] \cdot \begin{bmatrix} b_1^T \\ \vdots \\ b_{784}^T \end{bmatrix}$$

$U \in \mathbb{R}^{1000 \times 784}$ $B \in \mathbb{R}^{784 \times 784}$

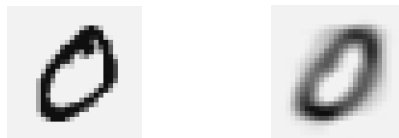
First 5 rows of B



- Reconstruction with $\hat{A} = [u_1 \ \cdots \ u_k] \cdot \begin{bmatrix} b_1^T \\ \vdots \\ b_k^T \end{bmatrix}$ for the first k singular values
- $\in \mathbb{R}^{1000 \times k}$ $\in \mathbb{R}^{k \times 784}$

Original a_1

Reconstruction $\hat{a}_1$



**Introduction to
Probabilistic Machine
Learning**

Unit 7 – Linear Basis Function
Models

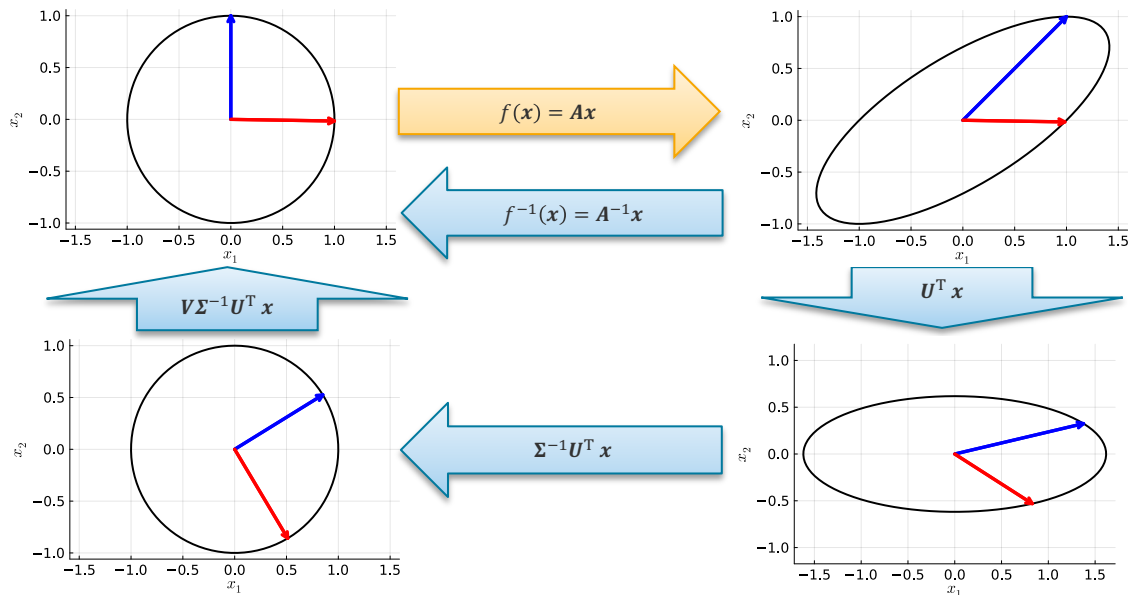
Inverse of a Matrix and SVD

- **Inverse.** The inverse A^{-1} of a full-rank square matrix A has the property that

$$A^{-1}A = AA^{-1} = I$$

- One way to compute it is with the singular value decomposition

$$A^{-1} = V\Sigma^{-1}U^T$$



**Introduction to
Probabilistic Machine
Learning**

Unit 7 – Linear Basis Function
Models

Cholesky Decomposition

- **Numerical Challenge:** Given a symmetric, positive-definite matrix $A \in \mathbb{R}^{n \times n}$ and a vector $b \in \mathbb{R}^n$ find the solution x such that

$$Ax = b$$

- **Naïve Solution:** Invert the matrix A and compute

$$x = A^{-1}b$$

- **Challenges:** If A has some singular values close to zero, this is numerically unstable!

- **Cholesky Decomposition:** Every symmetric positive-definite matrix $A \in \mathbb{R}^{n \times n}$ has a unique decomposition into a lower-triangular matrix $L \in \mathbb{R}^{n \times n}$

$$A = LL^T$$

- **Advantage:** Finding y such that $Ly = b$ can be done in $O(n^2)$ without an inverse simply using back-substitution (written as $y = L \backslash b$)!

- **Cholesky Solution:** Find the solution x for $Ax = b$ is a two-step algorithm

1. Compute $y = L \backslash b$
2. Compute $x = L^T \backslash y$

$$Ax = b \Leftrightarrow LL^T x = b$$

$$\downarrow$$

$$Ly = b$$



André-Louis Cholesky
(1875 – 1918)

$$\begin{pmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

$$y_1 = \frac{b_1}{L_{11}}$$

$$y_2 = \frac{b_2 - L_{21} \cdot y_1}{L_{22}}$$

$$y_3 = \frac{b_3 - L_{31} \cdot y_1 - L_{32} \cdot y_2}{L_{33}}$$

Cholesky Decomposition (ctd)

$$LL^T = \begin{pmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{pmatrix} \cdot \begin{pmatrix} L_{11} & L_{21} & L_{31} \\ 0 & L_{22} & L_{32} \\ 0 & 0 & L_{33} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{21} & A_{31} \\ A_{21} & A_{22} & A_{32} \\ A_{31} & A_{32} & A_{33} \end{pmatrix}$$

1 $L_{11}^2 = A_{11}$

$$L_{11} = \sqrt{A_{11}}$$

2 $L_{11} \cdot L_{21} = A_{21}$

$$L_{21} = \frac{A_{21}}{L_{11}}$$

3 $L_{21}^2 + L_{22}^2 = A_{22}$

$$L_{22} = \sqrt{A_{22} - L_{21}^2}$$

4 $L_{11} \cdot L_{31} = A_{31}$

$$L_{31} = \frac{A_{31}}{L_{11}}$$

5 $L_{21} \cdot L_{31} + L_{22} \cdot L_{32} = A_{32}$

$$L_{32} = \frac{A_{32} - L_{21} \cdot L_{31}}{L_{22}}$$

6 $L_{31}^2 + L_{32}^2 + L_{33}^2 = A_{33}$

$$L_{33} = \sqrt{A_{33} - L_{31}^2 - L_{32}^2}$$

```
function cholesky(A::Matrix)
    # check that the matrix is square
    if (size(A)[1] != size(A)[2])
        error("matrix must be square")
    end
    n = size(A)[1]

    # create a zero matrix
    L = zeros(n, n)

    # run the Cholesky decomposition
    for i = 1:n
        for j = 1:i
            sum = 0
            for k = 1:(j-1)
                sum += L[i, k] * L[j, k]
            end
            L[i, j] = (i == j) ? sqrt(A[j, j] - sum) : (A[i, j] - sum) / L[j, j]
        end
    end
    return (L)
end
```

■ Modelling Data

- Both textual and image data can be represented at different levels of granularity
- Images should ideally be represented in terms of feature of location and frequency: Gabor wavelets/filters are excellent candidate functions for such features

■ Linear Basis Functions Models

- Non-linear prediction models can be formed with non-linear basis functions
- Linearity is in the parameters, *not* the input dimensions of the data

■ Linear Mappings and Matrices

- All linear mappings can be expressed via matrix products
- The singular value decomposition is the rotation-scaling-rotation view on a mapping
- The Cholesky decomposition is numerically stable to help solve for the parameters of a linear model and is crucial for solving least-squares problems

**Introduction to
Probabilistic Machine
Learning**

*Unit 7 – Linear Basis Function
Models*

See you next week!